Full Stack Development with MERN
DarshanEase Project Documentation

1. Introduction

Project Title: DarshanEase

Team Members: Akshitha – Frontend Developer
Nanda Kishor – Backend Developer.

DarshanEase is a full-stack MERN application developed to simplify the process of booking darshan tickets online. The system enables devotees to browse temples, view available darshan slots, book tickets, make donations, and manage their bookings from a single platform. The application also provides a dedicated administration module for managing temples, slots, bookings, donations, and uploaded temple images.

2. Project Overview

Purpose: The project aims to digitize temple darshan booking by replacing manual booking with an online system that is secure, easy to use, and accessible.

Features:
• User Registration and Login using JWT authentication.
• Forgot Password and Profile Management.
• Browse temples and view detailed information.
• Book darshan slots and manage bookings.
• Online donation module.
• Admin Dashboard with CRUD operations for temples and slots.
• Booking and donation management.
• Temple image upload using Multer.
• Responsive React interface.

3. Architecture

Frontend: Built using React.js with React Router for navigation, Axios for API communication, and CSS for responsive UI. Components are separated into Users, Admin, Services, Components, Assets, and Styles folders.

Backend: Developed with Node.js and Express.js using REST APIs. Controllers process requests, routes define endpoints, middleware handles authentication and uploads, and Mongoose manages MongoDB interactions.

Database: MongoDB stores collections for Users, Temples, DarshanSlots, Bookings, and Donations. Relationships between collections are maintained using ObjectId references.

4. Setup Instructions

Prerequisites: Node.js, npm, MongoDB, VS Code.

Installation:
1. Clone the repository.
2. Run **npm install** in frontend and backend folders.
3. Configure the .env file with MongoDB URI, JWT Secret, and Port.
4. Start MongoDB.
5. Run backend using **npm run dev**.
6. Run frontend using **npm run dev**.

5. Folder Structure

Client: Components, Users, Admin, Services, Styles, Assets.
Server: config, controllers, middleware, models, routes, uploads, app.js and server.js.
The structure separates presentation, business logic, APIs, and database operations for maintainability.

6. Running the Application

Backend:
cd backend
npm run dev

Frontend:
cd frontend
npm run dev

7. API Documentation

Authentication APIs: Register, Login, Forgot Password.
Temple APIs: Get All, Get One, Add, Update, Delete.
Slot APIs: Create Slot, Get Slots, Get Slots by Temple.
Booking APIs: Create Booking, User Bookings, Temple Bookings, Cancel Booking.
Donation APIs: Create Donation, View Donations.
Admin APIs: Dashboard statistics.

8. Authentication

JWT-based authentication is implemented. After successful login, the backend returns a token and user details. Protected routes are accessible only to authenticated users, while role-based access can be used to restrict administrative features.

9. User Interface

The application provides responsive pages including Home, Login, Registration, Temples, Temple Details, Booking, Payment, Donation, Profile, My Bookings, Admin Dashboard, Manage Temples, Manage Slots, Bookings, and Donations.

10. Testing

Functional testing was performed for registration, login, CRUD operations, bookings, donations, authentication, image upload, and admin operations. API endpoints were verified and MongoDB data was validated after each operation.

11. Screenshots or Demo

https://drive.google.com/file/d/18pRkkuk60QrkejGnzFsxRzvwDi_DXQJr/view?usp=drivesdk

12. Known Issues

Online payment gateway is not integrated. Email notifications and deployment-specific configurations can be further enhanced.

13. Future Enhancements

Integrate payment gateway, email notifications, QR-code tickets, analytics dashboard, advanced search and filtering, cloud image storage, mobile application, and cloud deployment.